Final Project

2020110687 徐菱

2022 年 6 月 21 日

Problem 1 Please set up an ergodic Markov chain with AT LEAST three states. (a) Find the stationary probability for each state. (b) Show by a Monte Carlo method that the Markov chain has the stationary probability distribution.

Solution 1.1 Ergodic MC means irreducible, aperiodic and positive recurrent. The graph below shows an ergodic MC with 3 states.

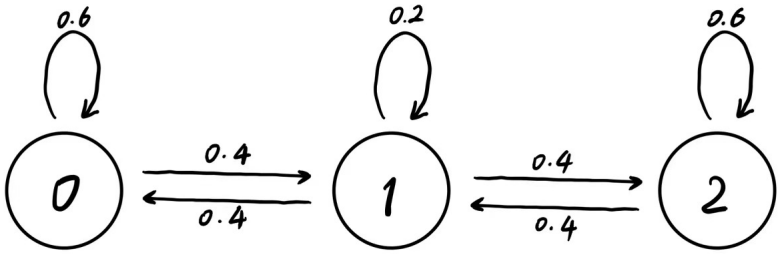


图 1: State Transition Diagram

Transition probability matrix is

$$P = \begin{pmatrix} 0.6 & 0.4 & 0 \\ 0.4 & 0.2 & 0.4 \\ 0 & 0.4 & 0.6 \end{pmatrix}$$

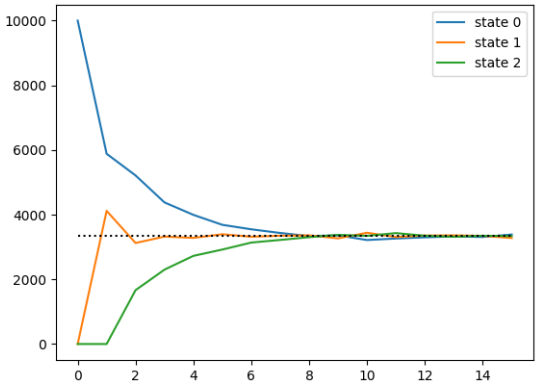
Stationary probability can be solved by

$$(P^T - I)x = 0 \Leftrightarrow x = (\frac{1}{3}, \frac{1}{3}, \frac{1}{3})^T$$

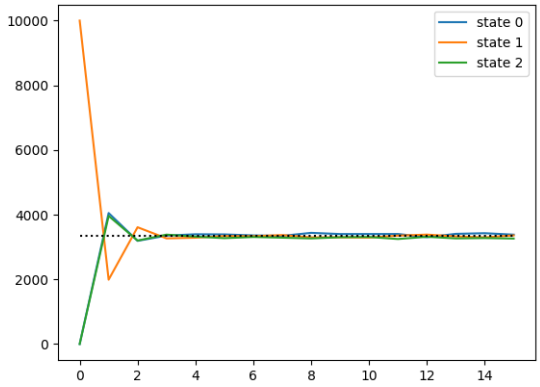
$$i.e. \pi_0 = \pi_1 = \pi_2 = \frac{1}{3}$$

Solution 1.2 Theoretically,the MC should has stationary probability distribution because it is ergodic. This can be proved by Monte Carlo method. In the python program Appendix 1, 1000 particles with initial state 0 move as the MC described. After 10 steps, 323 particles are in state 0, 335 in state 1, and 342 in state 2. This turns out to be quite close to 1:1:1.

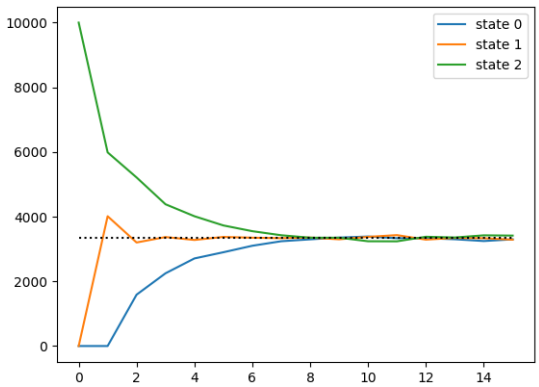
The program Appendix 2 shows how stationary probability converges. The 10000 particles take 15 movements, and the graph below shows the number of particles in each state at each step. Also, by changing the initial states, it can be proved that the stationary probability distribution is indepedent of initial state.



(a) start from 0



(b) start from 1



(c) start from 2

图 2: Proof of Convergency

Problem 2 By a Monte Carlo method, prove the following equality:

$$\int_0^T W_s dW_s = \frac{W_T^2}{2} - \frac{T}{2}$$

Solution 2 Assume that T=10. Denote

$$I_f = \frac{W_T^2}{2} - \frac{T}{2}$$

$$I_{f_n} = \sum_{k=0}^{n-1} W_{t_k}(W_{t_{k+1}} - W_{t_k})$$

$$t_k = \frac{k}{n}T, k = 0, 1, ..., n-1$$

By definition

$$W(0) = 0,\ W(t) \sim N(0, t),\ W_{t_{k+1}} - W_{t_k} \sim N(0, \frac{T}{n})$$

The equation to be proved equals

$$I_{f_n} \to I_f, a.s., in\ L_2$$

$$i.e.\ \lim_{n\to\infty} E(I_f - I_{f_n})^2 = 0$$

In the python program Appendix 3, the expectation is calculated by the average of 1000 samples, and the number of intervals is chaged between 1 to 100 . $E(I_{f_n})$ and $E(I_f)$ are always close to 0, but $E(I_f - I_{f_n})^2$ relates to number of intervals n. As $n \to \infty$, $E(I_f - I_{f_n})^2 \to 0$.

The figure below shows that $E(I_f - I_{f_n})^2$ converges to 0 quickly, as n increase. In fact, when $n \geq 50$, the difference between $E(I_f - I_{f_n})^2$ and 0 is neglectable. There are some fluctuation on $E(I_{f_n})$ and $E(I_f)$, but $E(I_f - I_{f_n})^2$ is very smooth.

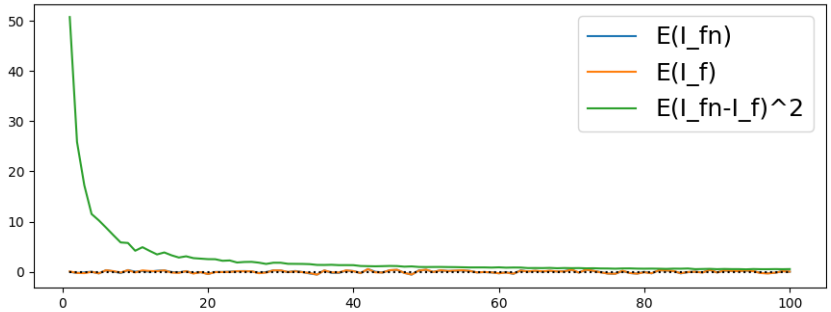


图 3: Convergency of I_{f_n}

Problem 3 By a Monte Carlo method, generate sample paths of the stock price following the geometric Brownian motion

$$\frac{dSt}{St} = rdt + \sigma dWt$$

where r is the risk-free interest rate and σ is the volatility of expected return. Based on your sample paths, calculate the price of a European call option by the risk-neutral formula:

$$E[e^{-rT} max\{S_T - K, 0\}]$$

and compare it to the result obtained by the Black-Scholes formula. You need to choose appropriate values for the model parameters ($r, \sigma, T, K \; and \; S_0$) first.

Solution 3 Assumptions of constants are as below

Symbol	Value	Meaning
r	1.5%	Risk-free interest rate
σ	30%	Volatility
T	1 year(250 trading days)	Due time
K	1000	Exerciseor strike price
S_0	1000	Initial price

表 1: Assumptions of Constants

Since the stock price obeys geometric Brownian motion,

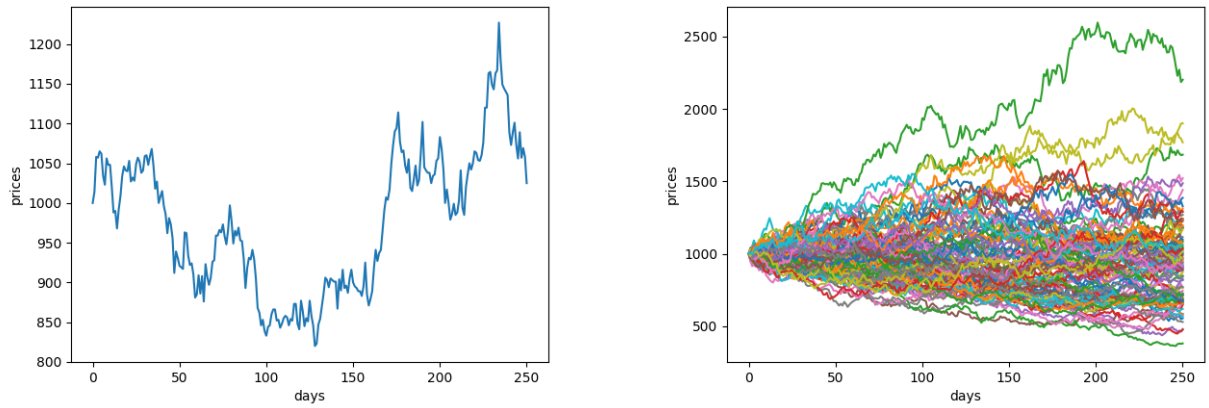
$$S_t = S_0 e^{(r-\sigma^2/2)t + \sigma W_t}$$

$$E(S_t) = S_0 e^{rt}, Var(S_t) = S_0^2 e^{2rt}(e^{\sigma^2 t} - 1)$$

In program Appendix 4, stock price at period t is calculated by

$$S_t = S_{t-1} e^{(r-\sigma^2/2)/T + \sigma W_{1/T}}, t = 1, 2, ..., T$$

where $W_{1/T} \sim N(0, \frac{1}{T})$. This method can generate a possible stock price path, as *figure* (a) shows below. By applying the method over and over again, more possible paths can be generated. Then the prices will look like *figure* (b).



(a) One of Possible Stock Price Paths

(b) 100 Possible Paths

图 4: Proof of Convergency

Then use the risk-neutral formula to calculate the estimated price. Program Appendix 5 generates 1000 sample paths, calculates S_T and $e^{-rT}max\{S_T - K, 0\}$, and uses sample mean to approximate expectation.

Also, use the formula of Black-Scholes Option Pricing Model to calculate the estimated price. Note that in these formulas $T = 1$.

$$c = S_0\Phi(d_1) - Ke^{-rT}\Phi(d_2)$$

$$d_1 = \frac{ln(S_0/K) + (r + \sigma^2/2)T}{\sigma\sqrt{T}}$$

$$d_2 = \frac{ln(S_0/K) + (r - \sigma^2/2)T}{\sigma\sqrt{T}} = d_1 - \sigma\sqrt{T}$$

By the former method, the estimated price is 128.076. While by the later one, it should be 125.939. The difference is about 1.697%, which is just acceptable.

However, using Monte Carlo method can be imprecise.Since $Var(S_T)$ is constant, the variance of $E[e^{-rT}max\{S_T - K, 0\}]$ may decrease at $O(\frac{1}{n})$, which is quite slow. Consequently, variance reduction technique is necessary.

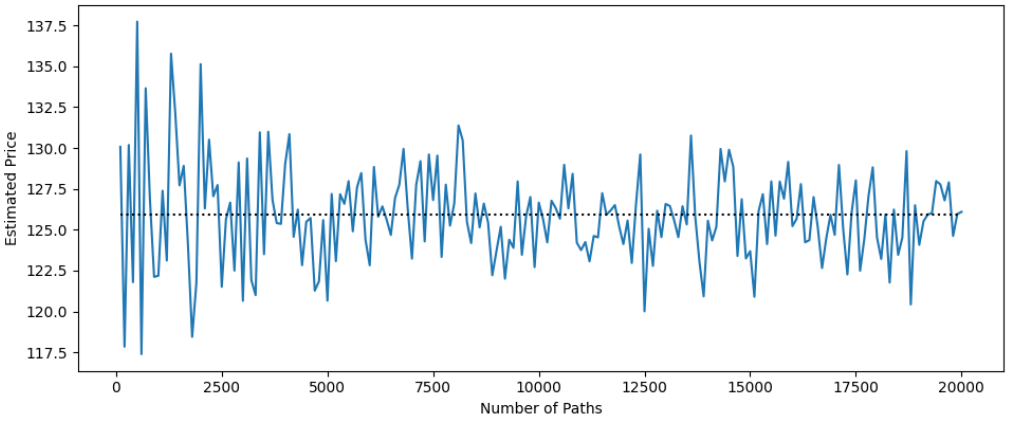


图 5: Convergency of Estimated Price

This article is typeset by LaTeX, using Visual Studio Code.

All the programs are written in Python 3.8, compiled by Pycharm.

No additional data used.

A Appendix 1

```
import numpy as np
import matplotlib as plt
from math import sqrt
from scipy.stats import norm
```

```
trans = np.array([[0.6, 0.4, 0], [0.4, 0.2, 0.4], [0, 0.4, 0.6]])  #
                                                    Transition probability
k = 0  # Initial state
time = 10  # Number of movements
num = 1000  # Number of initial states
frequency = [0, 0, 0]

for j in range(num):
    for i in range(time):
        k = int(np.random.choice(3, 1, p=trans[k]))
    frequency[k] += 1

print(frequency)
```

B Appendix 2

```
trans = np.array([[0.6, 0.4, 0], [0.4, 0.2, 0.4], [0, 0.4, 0.6]])  #
                                                    Transition probability
time = 15  # Number of movements
num = 10000  # Number of initial states
data = np.zeros([time+1, 3])
data[0][0] = num  # Initial state

def add_rand(i, rand):
    data[i][0] += np.sum(rand == 0)
    data[i][1] += np.sum(rand == 1)
    data[i][2] += np.sum(rand == 2)

for i in range(1, time+1):
    rand_1 = np.random.choice(3, int(data[i-1][0]), p=trans[0])
    rand_2 = np.random.choice(3, int(data[i-1][1]), p=trans[1])
    rand_3 = np.random.choice(3, int(data[i-1][2]), p=trans[2])
    add_rand(i, rand_1)
    add_rand(i, rand_2)
    add_rand(i, rand_3)

print(data)

x = np.linspace(0, time, time+1)
y_1 = data.T[0]
y_2 = data.T[1]
y_3 = data.T[2]
y_4 = np.full(time+1, num/3)  # Guide line
plt.plot(x, y_1, label="state 0")
plt.plot(x, y_2, label="state 1")
plt.plot(x, y_3, label="state 2")
plt.plot(x, y_4, color='k', linestyle=':')
plt.legend()
plt.show()
```

C Appendix 3

```
T = 10
n_max = 100  # Number of interval
tries = 1000  # Number of sample

def f(n):
    W = 0
    add = 0
    for i in range(n):
        W_ = np.random.normal(0, sqrt(T/n)) + W
        add += W * (W_ - W)
        W = W_
    return add, W

def E(n):
    sum_1 = 0  # To calculate E(I_fn)
    sum_2 = 0  # To calculate E(I_f)
    sum_3 = 0  # To calculate E(I_fn-I_f)^2
    for i in range(tries):
        k = f(n)
        sum_1 += k[0]
        sum_2 += k[1] ** 2 / 2 - T / 2
        sum_3 += (k[1] ** 2 / 2 - T / 2 - k[0]) ** 2
    return [sum_1 / tries, sum_2 / tries, sum_3 / tries]

x = np.linspace(1, n_max, n_max)
y = E(1)
for i in range(2, n_max+1):
    y = np.c_[y, E(i)]
plt.plot(x, y[0], label="E(I_fn)")
plt.plot(x, y[1], label="E(I_f)")
plt.plot(x, y[2], label="E(I_fn-I_f)^2")
plt.plot(x, np.full(n_max, 0), color='k', linestyle=':')
plt.legend()
plt.show()
```

D Appendix 4

```
# Constants
r = 0.015
sigma = 0.3
T = 250
K = 1000
S0 = 1000
num = 100  # Number of paths

data = np.full((T+1, num), S0)
for j in range(T):
    for i in range(num):
        data[j+1][i] = data[j][i]*np.exp((r-sigma**2/2)/T+sigma*np.random.
                                                    normal(0, sqrt(1/T)))

x = np.linspace(0, T, T+1)
for k in range(num):
    plt.plot(x, data.T[k])
plt.xlabel('days')
plt.ylabel('prices')
plt.show()
```

E Appendix 5

```
r = 0.015
sigma = 0.3
T = 250
K = 1000
S0 = 1000
num = 1000  # Number of paths

# Calculate by Monte Carlo Method
data = np.full(num, S0)
for i in range(num):
    data[i] *= np.exp((r-sigma**2/2)+sigma*np.random.normal(0, 1))
    data[i] = np.exp(-r)*max(data[i]-K, 0)
price_1 = sum(data) / num
print(price_1)

# Calculate by Pricing Fomula
d1 = (np.log(S0/K) + r+sigma**2/2) / sigma
d2 = d1 - sigma
price_2 = S0 * norm.cdf(d1) - K * np.exp(-r) * norm.cdf(d2)
print(price_2)
```